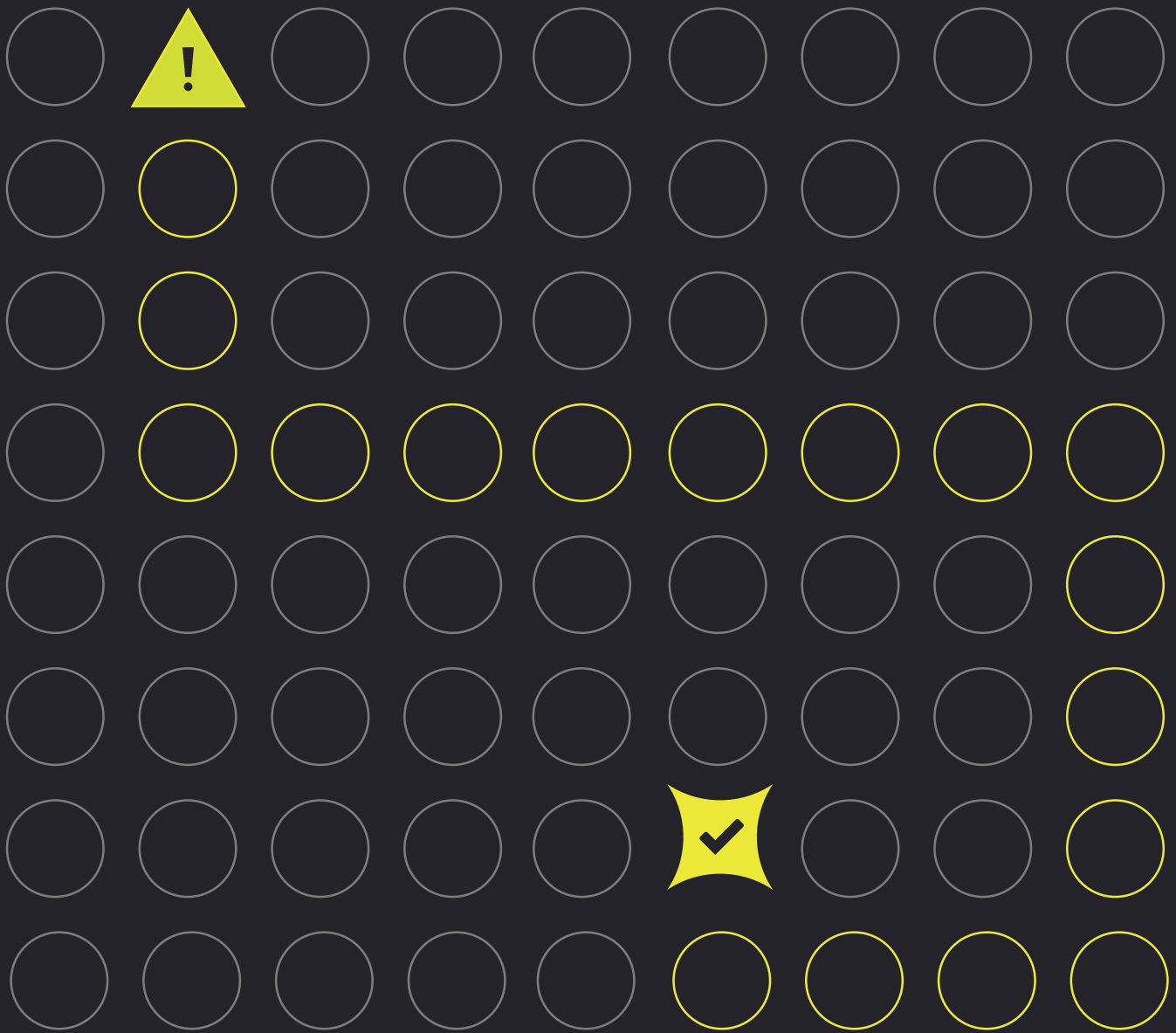




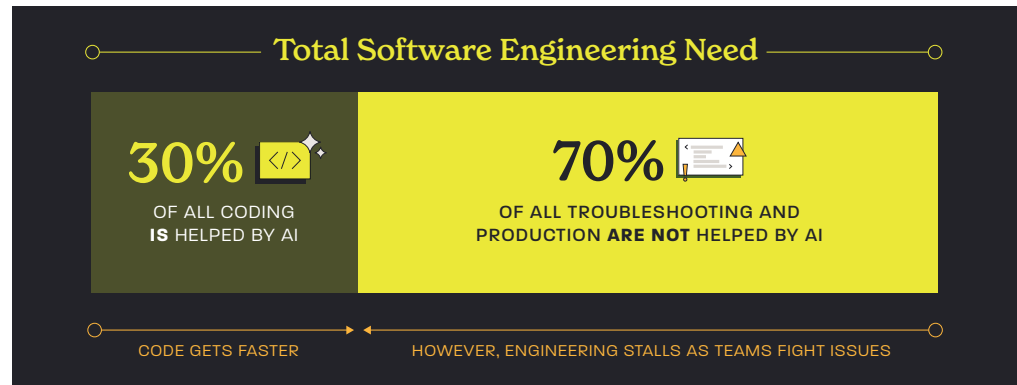
AI SRE Buyers Guide

From Strategy to Selection

 Resolve.ai

Executive Summary

Software engineering has reached an inflection point. While AI-powered code generation tools have accelerated development cycles, they address only 30% of engineering work: the coding itself. The remaining 70% lies in production operations: deploying code safely, maintaining system reliability, responding to incidents, and learning from failures. This imbalance creates a productivity paradox: code gets faster, but engineering velocity stalls because teams spend the majority of their time firefighting production issues rather than building new capabilities.



The Business Impact is Substantial

- Downtime and degradation cost the Global 2000 approximately \$400 billion annually ¹
- 54% of significant outages exceed \$100,000 in cost, with 20% surpassing \$1 million ²
- Only 16% of developer time is spent writing code, with the rest consumed by operational work ³
- War rooms and incident response drain senior engineering capacity, stalling strategic initiatives

AI Site Reliability Engineering (AI SRE) represents the next critical application of AI in software engineering: extending intelligence from code creation into production operations. Purpose-built AI SRE systems use multi-agent architectures to correlate code, infrastructure, and telemetry, providing real-time root cause analysis, prescriptive remediation, and continuous learning from every incident and human interaction.

This guide provides a decision framework across three critical dimensions:

- 1. Strategic Rationale:** Why AI SRE is now essential to engineering productivity and business resilience
- 2. Build vs. Buy Analysis:** The hidden costs and complexity of building versus buying AI SRE capabilities
- 3. Evaluation Criteria:** How to assess AI SRE solutions across both war-time (incident response) and peace-time (day-to-day operations) scenarios

Key findings for engineering leaders

- 95% of custom enterprise AI initiatives fail to reach production ⁴
- Gartner predicts 40% of agentic AI projects will be abandoned or re-architected by 2027 ⁵
- Teams that strategically apply AI across the full SDLC will achieve 25-30% productivity gains by 2028, nearly triple the impact of code-focused tools alone ⁶

The decision is not whether you need AI SRE, but how to implement it effectively. This guide helps you navigate that choice with clarity and confidence.

¹ Oxford Economics, "The Cost of Downtime & Service Degradation to the Global 2000," 2024

² Uptime Institute, "Annual Outage Analysis 2025," 2025

³ IDC, "Developer Time Allocation Study," 2024

⁴ MIT Sloan Management Review, "MLQ State of AI in Business 2025 Report," 2025

⁵ Gartner, "Agentic AI Initiative Outcomes: 2027 Predictions," 2024

⁶ Gartner, "AI Across the Software Development Lifecycle: Productivity Impact Analysis," 2024

Chapter 1: Why AI SRE Now

The Productivity Paradox

Generative AI has transformed software development. Code generation tools from companies like GitHub, Anthropic, and others have accelerated development cycles, shortened feedback loops, and removed friction from everyday coding tasks. Organizations from Robinhood and JPMorgan Chase to Walmart, Microsoft, Coinbase, and Google have publicly documented broad adoption of AI agents in code development and review.

Yet a fundamental problem remains: coding was never the bottleneck.

IDC analysis reveals that developers dedicate only about 16% of their time to writing code.⁷ The remaining 84% is consumed by operational work, background tasks, meetings, and production firefighting. The world's most expensive engineering talent spends the majority of its time triaging incidents, navigating tool silos, and wrestling with workflows designed for a simpler era.

This sets the scene for the next era of AI in software development lifecycle, AI for production systems, starting with AI SRE.

Production Environments: The Real Bottleneck

Modern production environments are sprawling ecosystems of complexity. Cloud-native architectures, containerized workloads, and Kubernetes orchestration generate exponentially more telemetry, dependencies, and moving parts than traditional infrastructure. When failures occur, engineers navigate a fragmented landscape:



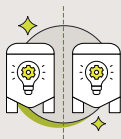
Tool sprawl:

Bouncing between Datadog, CloudWatch, Splunk, Loki, PagerDuty, Jira, Slack, and dozens of other systems



Context fragmentation:

Each tool uses its own query language, data format, and mental model



Knowledge silos:

Critical system understanding lives in the heads of senior engineers who remember "that time we had this issue before"

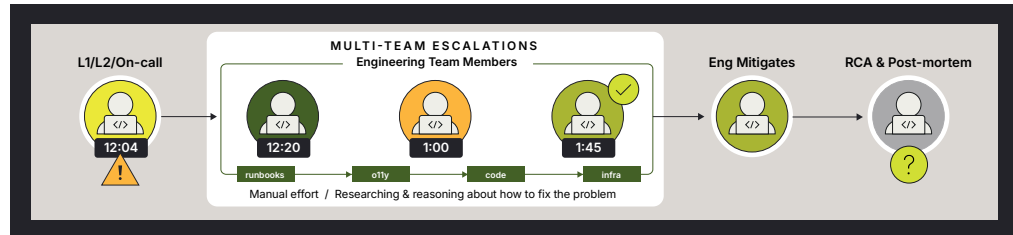


Legacy complexity:

Production systems accumulate migrations, technical debt, and heterogeneous on-prem, cloud, and SaaS environments

⁷ IDC, "Developer Time Allocation Study," 2024

When something breaks, the result is predictable: cascading war rooms where multiple teams converge, each with partial context, collectively burning hundreds of engineering hours to reconstruct what happened and why.



The business cost is staggering:

- Oxford Economics estimates downtime and service degradation cost the Global 2000 approximately \$400 billion annually⁸

For large organizations, downtime costs can reach **\$9,000 per minute**⁹

- Uptime Institute's 2025 analysis found that 54% of significant outages exceeded \$100,000, with 20% surpassing \$1 million, both figures increasing year-over-year¹⁰

Beyond major outages, frequent degradations and performance issues create an invisible tax on productivity. Every war room stalls roadmap work, increases context switching, and accelerates on-call burnout. What appears as "just a few hours of degraded service" compounds into thousands of lost developer hours annually.

The Hidden Tax

Major outages make headlines. **Daily degradations drain thousands of engineering hours** annually in war rooms, context switching, and burnout.

Why Legacy Approaches Cannot Solve Modern Reliability

Organizations have deployed automation, observability platforms, and Site Reliability Engineering practices for years. Runbooks automate known procedures. Dashboards surface metrics. APIs enable integration. CI/CD pipelines accelerate deployments.

Yet all share a fundamental limitation: they scale data and process, not understanding.

- Runbooks execute predefined steps but fail when confronted with novel failure modes
- Observability platforms generate metrics and logs but leave the cognitive burden on engineers to interpret what matters
- Traditional automation escalates alerts but provides no root cause analysis or remediation guidance
- Workflow tools route issues efficiently but cannot reason about system behavior

The outcome is predictable: more alerts, more dashboards, more logs, and more manual decisions. Legacy automation amplifies toil rather than eliminating it.

The AI SRE Paradigm Shift

AI has already proven transformative in software engineering. The 2025 Stack Overflow Developer Survey found that 84% of developers are using or plan to use AI tools, up from 76% the previous year.¹¹ But adoption without trust leads to abandonment. Engineers will not delegate production operations to AI unless it demonstrates transparency, reliability, and grounding in real systems.

⁸ Oxford Economics, "The Cost of Downtime and Service Degradation to the Global 2000," 2024

⁹ Industry analysis on enterprise downtime costs, 2024

¹⁰ Uptime Institute, "Annual Outage Analysis 2025," 2025

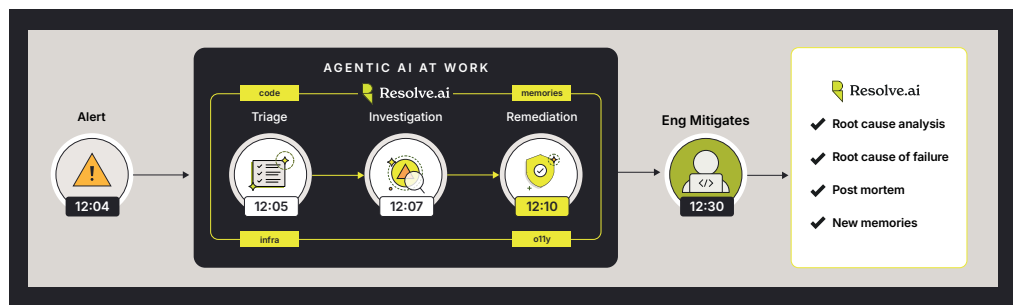
¹¹ Stack Overflow, "Developer Survey 2025," 2025

AI SRE fundamentally changes the equation. Purpose-built systems use large language models and multi-agent architectures to:

- Unify context across code repositories, infrastructure configuration, deployment history, telemetry data, and past incidents
- Generate narratives that explain what is happening in real-time, synthesizing disparate signals into coherent incident stories
- Identify root causes with supporting evidence, ranking hypotheses by confidence and showing the reasoning path
- Recommend remediation with prescriptive, actionable steps grounded in your specific environment
- Learn continuously from outcomes, feedback, and changing system topology

This relieves the heaviest cognitive burden on engineers: reconstructing what went wrong, understanding why it broke, and determining how to fix it. AI SRE does not replace engineers, it provides the same leverage AI brought to coding, applied to the complexity of production operations.

Instead of scaling data, it scales understanding at machine speed.



The Case for AI SRE is Wide Ranging

The case for AI SRE is **multi-prong**, impacting developer productivity, innovation, and key business metrics on costs, revenue, and customer satisfaction.

The Urgent Case for AI SRE

Several converging forces make AI SRE a strategic imperative today:

Financial pressure is intensifying. Uptime Institute's 2025 analysis shows outage costs rising year-over-year, with more incidents crossing the \$100,000 and \$1 million thresholds.¹² These figures capture only the most visible events. Day-to-day degradations carry equally damaging but less measurable costs in lost productivity and customer satisfaction.

- War rooms drain strategic capacity. Frequent performance issues and degradations pull senior engineers into reactive cycles. Each incident delays roadmap commitments, increases context switching overhead, and contributes to on-call burnout. The productivity cost compounds over time, creating technical debt in both systems and teams.
- Engineering time remains the scarcest resource. With only 16% of developer time spent writing code,¹³ the primary constraint on engineering velocity is operational complexity, not coding speed. Accelerating code generation without addressing production reliability delivers minimal ROI.

¹² Uptime Institute, "Annual Outage Analysis 2025," 2025

¹³ IDC, "Developer Time Allocation Study," 2024

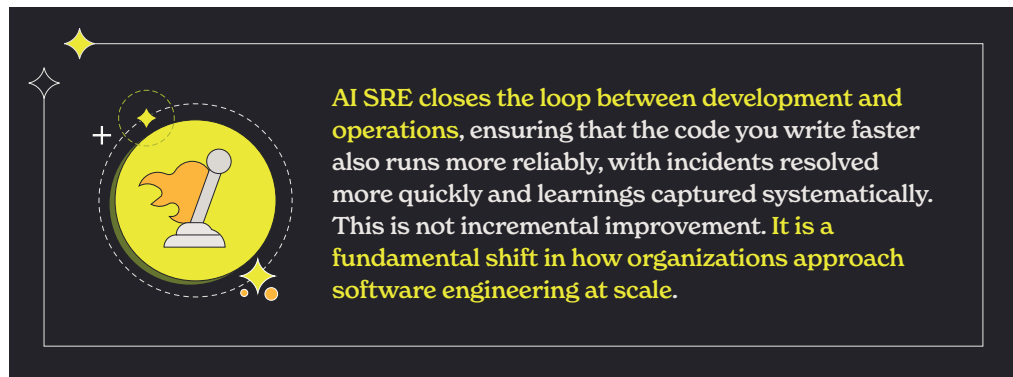
- Manual incident response does not scale. Each incident demands repetitive triage work: correlating logs, combing through hundreds of dashboards, checking recent deployments, checking recent deployments, examining metrics, searching historical incidents, and documenting findings. For distributed systems with dozens of services, this can consume hours of collective engineering effort per incident.
- AI adoption has reached mainstream. Cultural resistance to AI tooling has largely evaporated, with 84% of developers actively using or planning to adopt AI tools.¹⁴ The question is no longer whether to apply AI, but where it generates maximum leverage.
- Code assistants have hit a ceiling. Gartner research indicates most developers report productivity gains of 10% or less from AI code assistants. However, organizations that strategically apply AI across the full software development lifecycle, including production operations, will achieve 25-30% productivity gains by 2028, nearly tripling the impact of code-focused tools.¹⁵

Agentic AI represents the next wave. Gartner projects that software engineering agents, systems that plan, execute multi-step workflows, maintain context, and orchestrate across tools, will improve team productivity by 30-50% by 2028, far exceeding the 0-20% gains from current assistants.¹⁶ AI SRE embodies this vision in the most challenging domain: live production systems.

The Shift from Code Generation to Production Intelligence

The first wave of AI in software engineering delivered coding assistants that accelerate individual developer productivity. The second wave extends AI's leverage into production operations: from scaling code output to scaling system reliability and engineering velocity.

AI SRE closes the loop between development and operations, ensuring that the code you write faster also runs more reliably, with incidents resolved more quickly and learnings captured systematically. This is not incremental improvement. It is a fundamental shift in how organizations approach software engineering at scale.



¹⁴ Stack Overflow, "Developer Survey 2025," 2025

¹⁵ Gartner, "AI Across the Software Development Lifecycle: Productivity Impact Analysis," 2024

¹⁶ Gartner, "Software Engineering Agents: Market Projections 2024-2028," 2024

Chapter 2: Build vs. Buy — The Hidden Complexity of AI SRE

It's 2:13 a.m. Alerts fire. Your on-call engineer opens Slack, starts querying logs, and begins the familiar ritual: check dashboards, search for stack traces, review recent deployments, hope the issue matches something from memory. In this groggy, high-pressure state, engineers default to instinct rather than systematic reasoning.

This is precisely where agentic AI should shine, reasoning across distributed signals, correlating telemetry with code changes, and surfacing root causes backed by evidence, all before the engineer gets out of bed. With incidents costing millions annually and on-call engineers often requiring 6-9 months to reach full productivity, the case for AI SRE is urgent.

Yet most organizations attempting to build AI SRE capabilities in-house encounter a sobering reality: what looks achievable in a prototype becomes intractable in production.

The 5% Production Problem

Internal AI SRE initiatives typically start with one of three approaches:

- **Incident RAG systems:** Vector databases of past incidents connected to retrieval mechanisms, capable of summarizing ongoing issues or suggesting "we saw this before" guidance
- **Single-model orchestration:** Using Claude, GPT, or similar models with Model Context Protocol (MCP) servers, relying on custom prompts to chain tool calls
- **Hybrid implementations:** Combining retrieval databases with single-model orchestration, layering complexity without addressing fundamental architectural challenges

These approaches can demonstrate value in constrained environments, answering questions about past incidents, summarizing log patterns, or executing simple automation. But they collapse under the weight of production complexity.

Research confirms this pattern: MIT's MLQ State of AI in Business 2025 report found that only 5% of custom enterprise AI tools ever reach production.¹⁷ The gap between prototype and production-ready system is far wider than most organizations anticipate.

Why Prototypes Fail in Production

The disconnect between demo and deployment stems from fundamental architectural challenges.

Production environments are not static. Systems evolve daily through code deployments, infrastructure changes, dependency updates, and configuration modifications. An AI agent must maintain current understanding of services, dependencies, telemetry schemas, and operational patterns while adapting as these elements shift.

- Failure modes are not repetitive. While basic RAG systems excel at pattern matching against historical incidents, production failures often involve novel combinations of conditions. A new software version interacting with a specific infrastructure state under unusual load may produce symptoms never seen before. General-purpose models trained on broad corpora provide little advantage when facing proprietary system behaviors or infrastructure-specific failure modes.

Reality Check

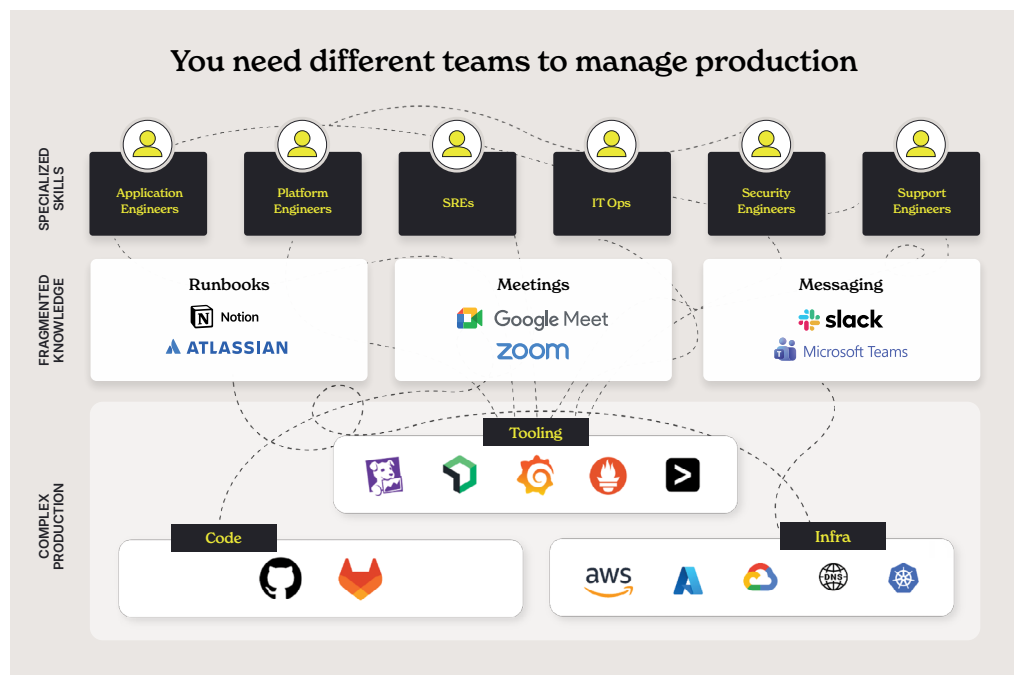
95% of custom AI tools never reach production.

The gap between "works in demo" and "works in production" is wider than most teams anticipate.

¹⁷ MIT Sloan Management Review, "MLQ State of AI in Business 2025 Report," 2025

- Model updates break stability. A new version of your foundation model ships with improved capabilities but subtly different behavior. Your carefully tuned prompts now produce inconsistent outputs. Agents that performed reliably last week now hallucinate or miss critical signals. Without systematic testing and model supervision, production systems become fragile.
- Data heterogeneity defeats simple approaches. A prototype tested against one Kubernetes cluster with consistent telemetry formats collapses when exposed to production's reality: multiple cloud providers, heterogeneous logging standards, legacy systems with different instrumentation, and partial observability across hybrid infrastructure.

The challenge is not purely technical, it is systemic. Building agentic systems that learn continuously, interpret shifting signals, and maintain reliability across evolving infrastructure requires both deep AI expertise and production systems domain knowledge.



What Production-Ready AI SRE Actually Requires

Effective AI SRE extends far beyond incident triage. It must support engineers across the full spectrum of production operations:

During incidents (war-time):

- Correlate telemetry across logs, metrics, traces, and events in real-time
- Form hypotheses about root causes and test them systematically
- Surface evidence chains that connect symptoms to underlying failures
- Recommend remediation steps grounded in system configuration and past outcomes
- Execute safe, pre-approved workflows within defined guardrails

During normal operations (peace-time):

- Answer contextual questions about system health, recent changes, and service dependencies
- Help engineers prepare for on-call shifts by summarizing unfamiliar services
- Investigate performance anomalies, cost spikes, or configuration drift before they become incidents
- Generate operational reports that synthesize system state for stakeholders
- Support vibe debugging: exploring subtle signals before they escalate into outages

Delivering this requires coordinating four technical capabilities within a multi-agent architecture:

1. **Knowledge:** Construct and maintain real-time understanding of your production environment: infrastructure topology, service dependencies, deployment history, code repositories, runbooks, and incident history. This is not static documentation but a living representation that updates as your systems evolve.
2. **Reasoning:** Formulate investigation plans, generate and test hypotheses, adapt strategies as new evidence emerges, and rank possible causes by confidence. Reasoning must be transparent, showing how conclusions were reached so engineers can validate, redirect, or extend investigations.
3. **Action:** Interact with production tools as engineers do: querying observability platforms, examining code repositories, reviewing deployment logs, and when appropriate, executing pre-approved remediation workflows like generating pull requests, adjusting service configurations, or triggering rollbacks.
4. **Learning:** Build persistent memory that improves over time, incorporating patterns from resolved incidents, feedback from engineers, and outcomes from remediation attempts. Learning must be continuous, not dependent on manual retraining cycles.

These agents must hand off structured outputs reliably. Without disciplined orchestration and robust error handling, multi-agent systems degrade into noise and hallucination.

The True Cost of Building In-House

Recent research underscores the difficulty of operationalizing agentic AI:

- McKinsey estimates generative AI's potential value in the trillions, yet finds only a fraction of enterprises deploying at scale^{19 20}
- MIT Technology Review reports that aligning LLMs often requires adversarial training to expose failure modes, highlighting inherent fragility²¹
- Gartner predicts that by 2027, 40% of agentic AI initiatives will be abandoned or re-architected due to performance issues²²

Building production-grade AI SRE requires strategic investment across three layers:

1. Foundation Layer:

- Systems-aware knowledge representation unifying documentation, telemetry, dependencies, and operational history into machine-readable substrates

¹⁹ McKinsey & Company, "Making Generative AI Production-Ready," 2024

²⁰ McKinsey & Company, "The Economic Potential of Generative AI," 2023

²¹ MIT Technology Review, "The Challenge of Aligning Large Language Models," 2024

²² Gartner, "Agentic AI Initiative Outcomes: 2027 Predictions," 2024

- Agent-specific CI/CD infrastructure with regression testing, golden datasets, and continuous evaluation frameworks
- Robust data pipelines that handle schema evolution, partial observability, and heterogeneous telemetry sources

2. Intelligence Layer:

- Post-training optimization including fine-tuning, reward model alignment, and continuous learning from implicit feedback
- Production-grade orchestration with memory management, multi-agent routing, context window optimization, and graceful degradation
- Model supervision systems that adapt to foundation model updates, API changes, and shifting production topology

3. Trust and Safety Layer:

- Instrumentation for confidence scoring, enabling agents to surface uncertainty and request human guidance appropriately
- Audit trails and explainability mechanisms that make every decision and action traceable
- Guardrails and approval workflows that define boundaries for autonomous actions

Even seemingly simple internal tools accrue hidden costs. Teams replacing vendor solutions with custom builds often achieve early wins but soon require extensive testing infrastructure, continuous integration for agent behavior, and persistent instrumentation to maintain trust. Multi-agent systems amplify these challenges exponentially.

The opportunity cost is substantial. A 2-3% improvement in engineering productivity translates to millions of dollars in value for most large organizations. Every week your senior engineers spend firefighting incidents, debugging brittle AI prototypes, or maintaining infrastructure is a week not spent on strategic initiatives that differentiate your business.

Opportunity Cost

A 2-3% productivity gain = millions in value.

Every week building infrastructure is a week not building your product.

When AI SRE Works: The Upside

When executed well, AI SRE delivers transformative impact. Consider these production scenarios:

Catching regressions before they become incidents. An AI SRE detects a creeping performance degradation: latency increasing by 50ms per hour across payment services. It correlates the change with a configuration update made three hours earlier, identifies the misconfigured rate limit causing connection pooling issues, and recommends the precise configuration rollback needed. The regression is caught and resolved before it impacts customers.

Tracing deployment failures in minutes. A service fails to deploy to production. Traditional debugging might consume hours bouncing between CI/CD logs, container registries, and Kubernetes events. An AI SRE traces the failure through the deployment pipeline, identifies a Dockerfile change that introduced an incompatible base image, and generates a pull request with the corrected configuration: all within minutes.

It's not about saving ten engineer-hours a week. It's about ensuring your best engineers aren't the only ones who can untangle your system at 2 a.m.

These are not demo scenarios. They represent production workflows where AI SRE extends engineering leverage across the full lifecycle, from development through operations.

The Strategic Decision Framework

For engineering leaders evaluating build versus buy, success begins with asking the right questions:

1. Strategic Value Assessment

Which application of agentic AI is core to your product or intellectual property? Will building it internally generate strategic differentiation that competitors cannot replicate, or are you recreating capabilities available in the market?

If AI SRE is not a core differentiator, internal development diverts scarce AI expertise from work that could set you apart. The question is not whether you can build it, but whether building it represents the highest-value use of your engineering talent.

2. Capability and Resource Reality Check

Do you have the depth of AI and production systems expertise required to build and maintain a multi-agent system? Building is not a one-time project, it demands ongoing investment in:

- Model supervision as foundation models evolve
- Continuous evaluation and testing infrastructure
- Post-training optimization and fine-tuning
- Integration maintenance as your production stack changes

Without a deep bench combining AI research skills and production operations expertise, you risk creating brittle systems that collapse under real-world complexity.

3. Opportunity Cost Analysis

Will this initiative accelerate or delay your broader engineering roadmap? Internal AI projects frequently redirect senior engineers from high-leverage work. The opportunity cost extends beyond headcount, it includes the innovation you defer by converting product engineers into platform maintainers.

Consider: if your AI talent built features that differentiate your core product instead of recreating infrastructure capabilities available in the market, what would the ROI difference be?

4. Value Optimization Strategy

Can you achieve 80% or more of the value faster through strategic partnerships? The most effective approach often involves buying foundational capabilities while customizing at the edge for your specific workflows and domain logic. This hybrid strategy delivers speed without sacrificing control over true differentiators.

Making the Call

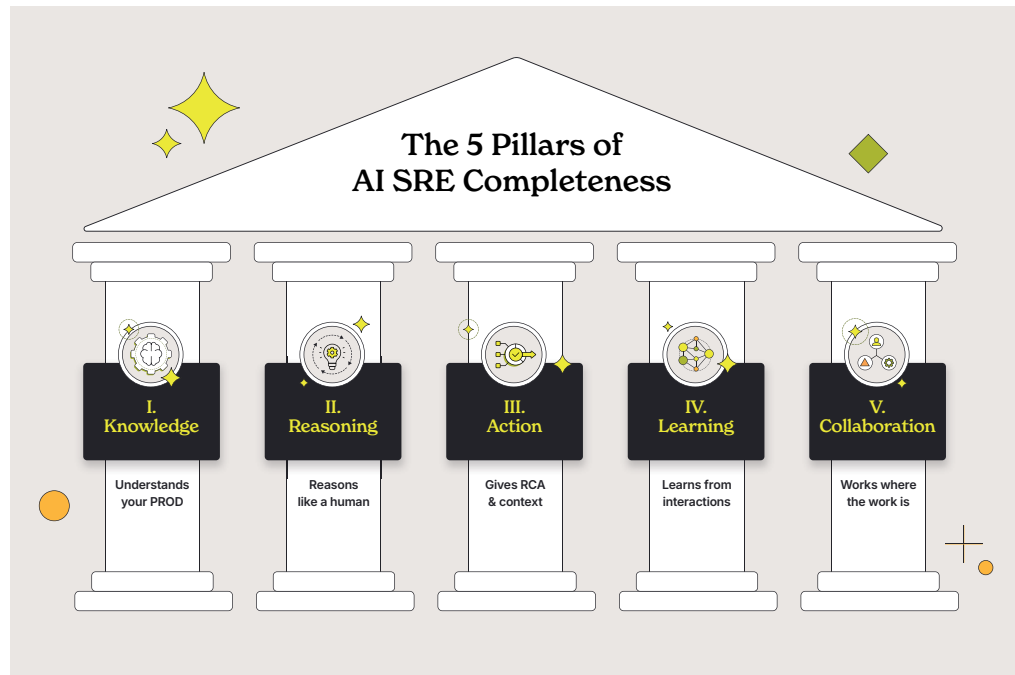
Agentic AI represents an architectural transformation in how software is designed, operated, and continuously improved. The question is not whether you can build it, most technically sophisticated organizations can assemble a working prototype. The question is whether you can build it well enough to deliver value that justifies the opportunity cost.

Most teams would not build a code generation model from scratch; these capabilities have become commoditized. A fully functional multi-agent system for production operations, by contrast, requires expertise at the intersection of AI research and systems engineering, ongoing investment as foundation models evolve, and continuous refinement as production environments change.

The reality is direct: DIY attempts at AI SRE frequently fail under production complexity, consume scarce technical talent that could drive competitive advantage elsewhere, and impose massive opportunity costs. Organizations that succeed align their build-versus-buy decisions with strategic priorities: building where it creates durable differentiation, buying where it accelerates time-to-value, and always optimizing for engineering leverage on their core mission.

Chapter 3: How to Evaluate AI SRE Solutions

This chapter is intentionally comprehensive. It is designed to expose how a candidate system behaves under real pressure and in everyday operations, not just in a curated demo. The goal is to reduce MTTR, improve root-cause quality, and reclaim engineering capacity in your environment. Use every section below; none are optional.



The Five Pillars of AI SRE Completeness

Building on the four technical capabilities outlined previously, these pillars distinguish production-ready platforms from sophisticated prototypes. A credible solution should demonstrate all five, in production, with your data.

1. Knowledge

- **What it means:** A continuously updated, system-aware graph of services, dependencies, deployment history, configurations, code, runbooks, and incident timelines. Not static docs. Not a one-time ingest.
- **How to verify:** Break your staging topology in a small way (config drift, new dependency) and see if the system's world model updates within minutes and reflects the change in investigations.
- **Red flags:** Manual refresh cycles; stale dependency maps; missing recent deployments; "it will learn that later" answers.

2. Reasoning

- **What it means:** Plans investigations, generates and tests hypotheses, reformulates when evidence contradicts the current path, and ranks alternatives by confidence with transparent logic.
- **How to verify:** Present two incidents with similar symptoms but different causes. Require the agent to branch and abandon incorrect hypotheses with clear citations.
- **Red flags:** Fixed flows; single-shot answers; confident but uncited claims; inability to show why a path was rejected.

3. Action

What it means: Executes read and write operations safely across your stack: Querying logs, traces, metrics, changelogs, and, when approved, issuing rollbacks, PRs, or config changes within guardrails.

How to verify: Define an approval-gated remediation (e.g., revert a feature flag). Require the system to propose, request approval, execute, and verify impact.

Red flags: Advice only; screen-scraping dashboards; no audit trail; no ability to verify outcomes.

4. Learning

- **What it means:** Performance improves from outcomes and feedback without manual retraining sprints. Patterns discovered during incidents reduce toil in future cases.
- **How to verify:** After a first incident class is resolved, trigger a near-duplicate. Look for faster diagnosis and earlier, more confident recommendations.
- **Red flags:** "We can fine-tune later"; no memory of past incidents; repeated mistakes.

5. Collaboration

- **What it means:** Works where engineers work (Slack, incident tools), explains steps, accepts redirection, and hands off cleanly.
- **How to verify:** Mid-investigation, ask the agent to change focus. Evaluate whether it preserves context, cites why it pivoted, and posts a coherent summary to the incident channel.
- **Red flags:** Context loss; chat-only UX; copy-paste into tickets; no single source of truth.

Why Early Evaluations Often Mislead

Demos hide the realities of production: noisy telemetry, novel failure combinations, model drift, schema changes, and time pressure. They select incidents with obvious signatures, pre-authorized data paths, and perfect instrumentation. Your evaluation must deliberately push into messy edges: partial logging, mixed cloud vendors, concurrent incidents, and teams using their own service names and jargon.

Test What Matters

Demos hide production reality: noisy telemetry, novel failures, schema drift, time pressure.

Push into the messy edges where systems actually break.

Required anti-demo tests

- **Noisy data:** Inject synthetic noise into one signal. The agent should down-rank it, not chase it.
- **Novelty:** Reproduce an issue with a new code version. The agent should not rely solely on historical matches.
- **Drift:** Change a telemetry schema mid-evaluation. The system should adapt without retraining.
- **Time pressure:** Cap investigation steps to 10 minutes and assess quality of the best available answer.

Structuring an Effective Evaluation

Scope and timeline

- Run for **2-4 weeks**. Include a minimum of **two live incidents** and **five historical incidents** across **three services**.
- Integrations in scope: **Observability (logs, metrics, traces); incident management; CI/CD; infrastructure; source control; knowledge bases; Slack.**

Decision Speed

2-4 weeks of real production testing tells you more than 6 months of pilots. **Test fast, decide faster.**

War time (incidents under pressure)

- Use real alerts. Require the agent to: identify likely root cause, cite exact evidence, name the right owner team first, and propose remediation with risk.
- Capture: time to first plausible root-cause hypothesis; time to evidence citation; time to validated fix or rollback.

Peace time (everyday operations)

- Daily usage by on-call engineers: health summaries, readiness briefs, dependency questions, “vibe debugging,” and cost or latency anomaly triage.
- Capture: time saved on on-call prep; frequency and usefulness of answers; proactive detection lead time for issues that did not page.

Decision checkpoints

- **End of week 2:** Integrations complete; one historical and one live incident exercised; preliminary MTTR delta.
- **End of week 4:** Repeatability proven across two services; agent autonomy expanded for read actions.
- **End of evaluation:** Final metrics, qualitative survey, and a go/no-go with next-step runbooks.

War Time Measures (what matters during incidents)

- **MTTR delta:** Percent reduction versus a 90-day baseline by severity.
- **Root-cause quality:** Presence of causal chain with citations; correctness validated post-incident.
- **First-team-right rate:** Percent of incidents sent to the correct owner on the first pass.
- **Evidence latency:** Minutes to first concrete citation (log line, metric, trace, commit).
- **Alert consolidation:** Reduction in duplicate or downstream alerts escalated to humans.
- **Handoff quality:** Clarity of the Slack or ticket summary for executive and cross-team audiences.

Peace Time Measures (everyday value between incidents)

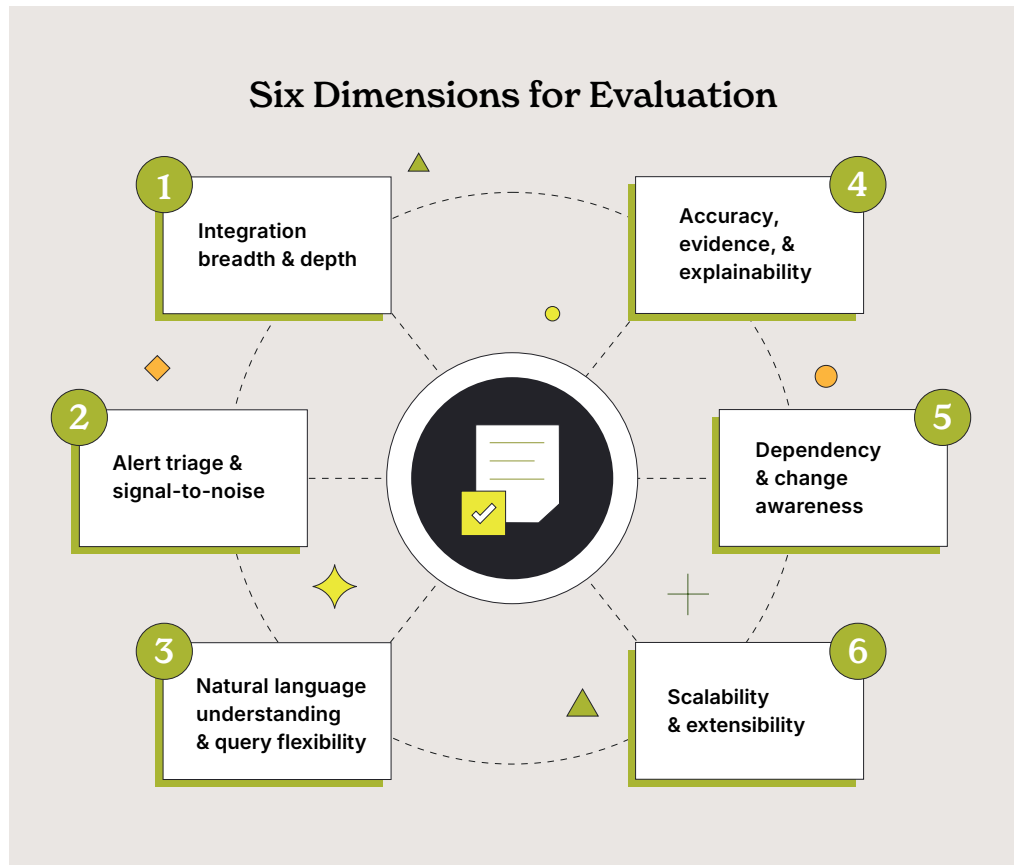
- **On-call prep time:** Reduction per shift for new and experienced engineers.
- **Operational questions answered:** Daily queries served with satisfactory ratings.
- **Detection lead time:** Minutes or hours of warning before the first page for regression classes.
- **Toil reduction:** Time saved on report generation, incident write-ups, and change risk briefs.
- **Knowledge capture:** New patterns and playbooks generated automatically from resolved incidents.

Balancing Metrics with Outcomes

Pick a compact set that drives decisions. Tie each metric to an action (e.g., expand autonomy, broaden rollout, or change alert routing).

Quantitative: MTTR; incidents without war rooms; engineers per incident; on-call prep time; peace-time query volume; SLA posture.

Qualitative: Alert fatigue; cross-team collaboration; onboarding speed; deployment confidence; reclaimed capacity for roadmap work.



Six Dimensions for Comprehensive Evaluation

Each dimension includes what good looks like, a test, and red flags.

1. Integration breadth and depth

- **Good:** Correlates a symptom in one tool with changes in CI/CD and infra state, end to end.
- **Test:** Present a metric spike in Datadog that originated in a config change in Terraform. Require a single narrative.
- **Red flags:** CSV exports; single-vendor lock; no infra awareness.

2. Alert triage and signal-to-noise

- **Good:** Clusters alerts into one incident story, preserving causal order.
- **Test:** Trigger a cascading failure across three services. Expect one root narrative, not three separate pages.
- **Red flags:** Summaries without causality; alert-per-alert replies.

3. Natural language understanding and query flexibility

- **Good:** Maps "checkout slow" to the correct services, metrics, and logs using your internal names.
- **Test:** Ask using product terms and team slang; expect accurate mapping and follow-ups that maintain context.
- **Red flags:** Rigid syntax; loss of context across turns.

4. Accuracy, evidence, and explainability

- **Good:** Every claim has a linkable artifact (log, metric, trace, code diff) and a visible reasoning trail.
- **Test:** “Show me the evidence.” Require inline citations and a short chain-of-logic summary.
- **Red flags:** Confident, generic advice; hallucinated details; no confidence expression.

5. Dependency and change awareness

- **Good:** Connects symptoms to recent deployments, feature flags, and upstream/downstream dependencies.
- **Test:** Roll a change to an upstream service. Expect immediate linkage when a downstream error rate moves.
- **Red flags:** No temporal awareness; misses change windows; ignores dependencies.

6. Scalability and extensibility

- **Good:** Handles concurrent users and high log volumes; adapts to schema changes; onboards new teams quickly.
- **Test:** Simulate 20 parallel investigations; update a log schema mid-run; add a new team and service in a day.
- **Red flags:** Slowdowns; broken parsers after changes; vendor PS required for every integration.

Enterprise Readiness: Beyond Technical Capabilities

- Security and compliance: Deployment options, data boundaries, SOC 2, GDPR, HIPAA mappings, key management.
- Governance and audit: Approval gates, role-based control, full action and reasoning audit trails.
- Workflow integration: Slack or incident tooling posts with artifacts; bi-directional ticket sync; PR or change control hooks.
- UI and transparency: Step-by-step rationale, drill-downs, confidence surfacing, and error-handling clarity.
- Operations at scale: HA architecture, SLOs, backup and recovery plans, and vendor on-call.
- References and outcomes: Similar-scale production references with measurable deltas on MTTR and toil.
- Roadmap clarity: Short-term commitments (next 2 quarters) and a principled path to bounded autonomy.

Evaluation in Practice: Concrete Examples

Example 1: Tool-aware investigation

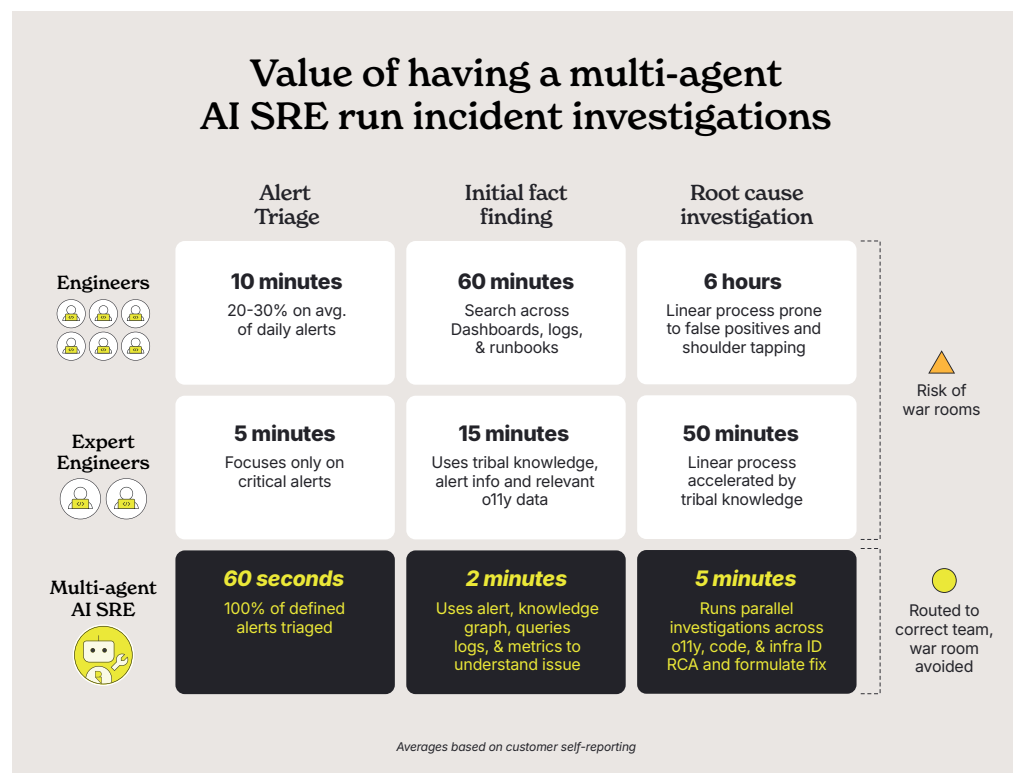
1. 03:12: storage 502s spike.
2. Agent translates the question into precise queries; auto-selects the correct time window.
3. Surfaces “TLS certificate expired” lines in load balancer logs and links to the expiry timestamp.
4. Cross-checks recent deployments to exclude regressions; proposes reissue with validation steps; verifies resolution.

Example 2: Multi-hop reasoning under pressure

1. 14:23: payment timeouts across services.
2. Agent correlates DB "max connections reached" with auth connection-pool exhaustion and payment timeouts.
3. Traces to a recent auth deployment; offers throttle, rollback, or DB scaling options with risks and expected impact.
4. Posts a succinct incident summary with evidence and a proposed path; updates after action taken.

Example 3: Peace-time operational support

1. On-call prep for payments.
2. Agent compiles current telemetry, active alerts, recent changes, and relevant incident history into a health brief.
3. Flags degraded dependencies and rate-limit risks; recommends checks and owners before the next shift.



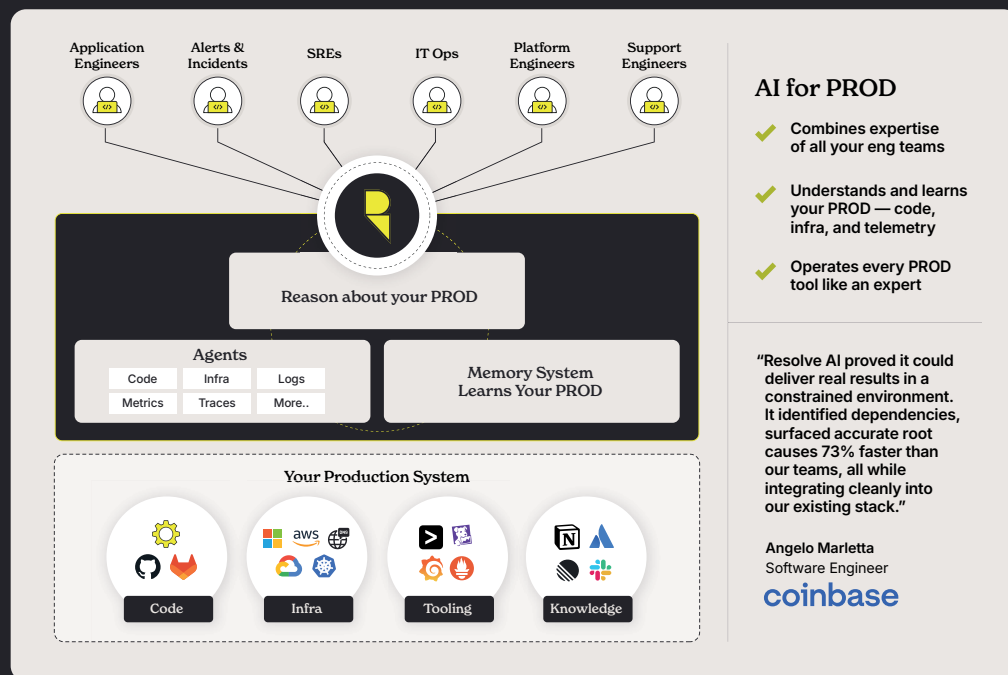
About Resolve AI

Resolve AI is AI for Production Systems. Resolve works across code, infrastructure, and telemetry to autonomously investigate incidents, help optimize costs, and ship faster. Companies like Coinbase, Zscaler, and Gametime use Resolve AI to run their production systems.

It is a purpose built, multi agent platform that combines four capabilities — knowledge, reasoning, action, and learning — in a multi agent architecture where specialized agents collaborate like engineers. Agents are fluent in tools such as Grafana, Datadog, and Splunk. They form and test hypotheses, query metrics, logs, and traces, and surface root causes rather than correlated symptoms. During an investigation, agents run parallel theories, rank them by confidence, and present transparent evidence and confidence. Engineers can redirect or validate investigations in Slack or in the UI, and every step is captured in an audit trail.

Resolve's memory system and knowledge graph learn from every incident and interaction. They maintain a real time view of services, dependencies, deployments, configuration, and prior incidents so investigations start with context, not guesswork. Teams use this to cut time to evidence, reduce war rooms, and turn tribal knowledge into durable practice that scales across users and teams.

Built by leaders in observability and applied AI, including early OpenTelemetry contributors and senior engineers from Splunk and top research groups, Resolve is designed to work in complex production environments with messy, incomplete telemetry. This is a critical differentiator where demo centric approaches typically fail. The system has been proven at large enterprises in high stakes applications, delivering time to value within days, not months.



Chapter 4: Making the Decision, A Framework for Action

The Strategic Decision Matrix

Use this matrix to translate analysis into action.

1. **Urgency assessment:** Quantify downtime cost, engineering burden, and customer impact.
2. **Readiness assessment:** Confirm observability maturity, incident processes, and sponsorship.
3. **Value optimization strategy:** Decide build, buy, or hybrid based on differentiation and capacity.
4. **Timeline and milestones:** Plan evaluation, pilot, rollout, and optimization with clear KPIs.

Buy vs Build Reality

Building AI SRE from scratch: 12-18 months, 10+ engineers, fragile in production. You'll generally always be behind the latest capabilities from leading providers.

Buying and customizing: 4-8 weeks, proven at scale, focus on your differentiators.

Building Internal Consensus

Tailor the message to each stakeholder group:

- **Engineering leadership:** Engineering leverage, roadmap execution, retention.
- **SRE and operations:** Force multiplier, knowledge capture, transparent control.
- **Application developers:** Peace time value, reduced context switching, deployment confidence.
- **Finance and executives:** Downtime reduction, customer trust, ROI versus code assistant gains.

Measuring Success Post Deployment

Track progress with consistent metrics.

- **Primary business:** MTTR, downtime minutes, SLA posture, incident cost.
- **Engineering efficiency:** Hours per incident, engineers per incident, war room frequency, on call ramp.
- **Adoption and trust:** Daily active users, peace time queries, satisfaction, influence on resolutions.
- **Learning and improvement:** Root cause accuracy, false positives, time saved, knowledge growth.

Target outcomes at 6 to 12 months:

- 25 to 30% MTTR reduction for P0 or P1 incidents.
- 50% reduction in war room escalations.
- 10 to 15% engineering capacity reclaimed.
- 85% plus engineer satisfaction with AI SRE utility.

Common Implementation Pitfalls to Avoid

1. Treating AI SRE as just another dashboard.
2. Over-constraining autonomous actions that do not modify the state.
3. Shallow integrations that lack access to raw telemetry.
4. Ignoring the peace-time value and daily usage.
5. Skipping feedback loops needed for continuous learning.
6. Expecting full autonomy instead of augmented intelligence.

Conclusion: From AI in Code to AI for Production

The story of AI in engineering is shifting. The first act accelerated coding. The decisive act is operationalizing AI across the lifecycle, planning, building, testing, deploying, operating, and learning, in one loop that improves with every change and every incident.

Reliability is a critical outcome, not the destination. The destination is a production intelligent SDLC, where systems understand their own state, where changes carry an explainable blast radius, where evidence travels with decisions, and where learning compounds across teams and time. In this model, AI is not a tool. It is the connective tissue that turns telemetry into judgment, judgment into action, and action into durable capability.

Leaders who make this shift will change the unit economics of software. They will ship faster with fewer regressions, recover sooner with clearer accountability, and redirect scarce engineering hours from toil to invention. Customers will feel it as quieter outages, steadier performance, and more credible promises. Teams will feel it as fewer war rooms, faster onboarding, and room to build what moves the business.

Treat AI in production as the connective layer above version control, CI/CD, observability, and cloud, unifying them into one loop of planning, building, testing, deploying, operating, and learning. Commit to three principles that separate leaders from laggards:

- Production, not prototypes: Prove value in live systems with messy data, and let those lessons drive the roadmap.
- Evidence, not opinions: Make every diagnosis and action explainable, auditable, and grounded in your environment.
- Learning, not one offs: Encode outcomes so each incident, change, and review raises the baseline for the next one.

Pragmatic path: Start by quantifying urgency, put a clear dollar figure on downtime, incident labor, and churn risk so leaders agree on the size of the problem. Confirm readiness next, make sure observability, incident process, and access patterns are good enough for a fair test. Then run a 2 to 4 week PoC in real conditions, include wartime and peacetime use, real integrations, and success criteria tied to MTTR and adoption. At the decision point, choose build, buy, or hybrid, build where it differentiates, buy where speed and proven depth matter, customize at the edge. Roll out in stages with a learning loop, start on the highest impact services, publish a simple monthly scorecard, and widen autonomy as trust, auditability, and outcomes improve.

